

PHASE 1 explanation :

Databases play a critical role in data analysis by providing structured storage, enforcing data integrity, and enabling efficient querying of large datasets using SQL. Unlike spreadsheets, which are limited in size and prone to duplication and inconsistency, databases support relational design, constraints, and multi-table queries that ensure accuracy and scalability. The Telco Customer Churn dataset is relational because it contains logically connected entities such as customers, services, billing details, and churn outcomes, which are linked through a common customer identifier and are best analyzed using relationships and joins.

PHASE 3: Data Manipulation, Querying & Schema Maintenance

1. DDL – Data Definition Language

What it is: DDL includes commands like CREATE, ALTER, and DROP. It's used to define or change the structure of the database.

When to use it:

When you need to create a new table, index, or view.

When you need to update the schema, like adding a column to store new data.

When cleaning up, such as dropping old tables that are no longer needed.

Real-world scenario:

Imagine a company wants to track customer feedback. You would use DDL to create a new Feedback table with columns like CustomerID, Rating, and Comment. Later, if you need to store feedback date, you might ALTER the table to add a Date column.

DDL creates the tables and relationships.

2. DML – Data Manipulation Language

What it is: DML commands (INSERT, UPDATE, DELETE) are used to manage the data inside tables without changing the structure.

When to use it:

To add new records, like a new customer signup.

To update existing records, for example correcting a wrong email.

To remove unnecessary data, such as deleting test data.

Real-world scenario:

In a sales database, each time a new order is placed, you would use INSERT to add it. If the customer changes their shipping address, you use UPDATE. If an order was canceled and should not remain in the active system, you use DELETE.

DML populates the tables with raw data.

3. DQL – Data Query Language

What it is: DQL is primarily the SELECT command, used to retrieve data for analysis, reporting, or validation.

When to use it:

To explore and analyze data.

To validate data after migration or cleaning.

To generate reports or summaries.

Real-world scenario:

A marketing team wants to know which products sold most last month. You would use a DQL SELECT query to pull sales data grouped by product. Or in a data pipeline, you may SELECT data to check if all required fields are populated, ensuring the system is ready for further processing.

DQL and metadata queries check the data, summarize it, and ensure it's ready for analytics.

Why metadata queries matter

Metadata is data about the data, like table names, column types, and constraints. Queries that explore metadata are essential because:

They help validate that your database has the expected structure.

They prevent errors when running scripts like trying to insert data into a missing column.

They provide documentation for teams to understand the database without guessing.

Example: Before running a cleaning pipeline, you might query the metadata to see which columns allow nulls or which tables have a PRIMARY KEY, so your updates don't break the schema.

PHASE 6: Subqueries & CTEs

Compare when CTE is better than a subquery

CTE is better than a subquery when:

- The query is long and complex, so dividing work into logical steps improves readability
- The result of the query is reused multiple times in the same statement
- When you want to make intermediate results easier to debug and maintain

PHASE 8: Views for Reporting

When to use Views vs CTEs:

Views are permanent, reusable, and useful for security and reporting

CTEs are temporary, better for complex queries, logical steps, or debugging

Benefits of views for reporting:

- It Simple complex joins and calculations
- Provide consistent dataset for multiple reports
- Can restrict access to sensitive columns for certain users (Good for defining roles)